

# RTL Design Sherpa

# Delta Hardware Architecture Specification 1.0

**October 14, 2026**

# Table of Contents

1 Delta Has Index....12

2 Document Information....13

2.1 Delta Hardware Architecture Specification....13

2.2 Revision History....14

2.3 Document Purpose....15

2.4 Intended Audience....16

2.5 Related Documents....17

3 Purpose and Scope....18

3.1 What is Delta?....18

3.2 The Problem Delta Solves....19

3.3 Scope....20

3.4 Out of Scope....21

3.5 Non-Goal: Memory-Mapped Protocol Conversion....22

4 Document Conventions....23

4.1 Naming Conventions....23

4.2 Signal Direction Conventions....24

4.3 Numbering Conventions....25

4.4 Typographic Conventions....26

4.5 Versioning Conventions....27

5 Definitions and Acronyms....28

5.1 Acronyms....28

5.2 Key Terms....29

6 Use Cases....30
6.1 Primary: RISC Cores + DSP Array....30
6.2 Secondary: Sensor/Peripheral Fabric....31
6.3 Tertiary: AMBA 5-Aware Streaming Interconnect....32
6.4 Educational Demonstration....33
6.5 Mesh-Mode: Monitored Manycore Fabric....34
7 Key Features....35
7.1 Flat M×N Crossbar....35
7.2 TDEST-Based Routing....36
7.3 Packet-Atomic Round-Robin Arbitration....37
7.4 Combinational Data Mux with Registered Grant....38
7.5 Optional Richness (Default Off)....39
7.6 Optional Packet Counters....40
7.7 Formal Assertions....41
7.8 Code Generation....42
7.9 Mesh-Mode Features....43
8 System Context....44
8.1 Where Delta Fits....44
8.1.1 Figure 2.1: Delta System Context....44
8.2 Upstream Requirements....45
8.3 Downstream Requirements....46
8.4 Clocking and Reset....47
8.5 Relationship to Other Components....48
8.6 Mesh-Mode Context....49

9 Block Diagram....50
9.1 Delta Architecture Overview....50
9.1.1 Figure 3.1: Delta High-Level Block Diagram....50
9.2 Functional Blocks....51
9.2.1 TDEST Decode....51
9.2.2 Per-Slave Round-Robin Arbiter....51
9.2.3 Crossbar Mux....51
9.2.4 TREADY Steering....51
9.2.5 TWAKEUP Steering (AMBA5, Optional)....51
9.2.6 Interface Checks (AMBA5, Optional)....51
9.2.7 Packet Counters (Optional)....51
9.3 Scalability....52
9.3.1 Resource Scaling....52
10 Data Flow....53
10.1 Request Flow....53
10.2 Payload Flow....54
10.3 Backpressure Flow....55
10.4 Wakeup Flow (AMBA5)....56
10.5 Packet Boundaries....57
11 Protocol Support....58
11.1 AXI4-Stream Base Flavor....58
11.2 Optional TSTRB Byte Qualifiers....59
11.3 Optional AMBA 5 Sidebands....60
11.4 What Delta Does Not Support....61

11.5 Protocol Compliance Summary....62
12 AXI-Stream Base Interface....63
12.1 Master Interface (S_AXIS)....63
12.2 Slave Interface (M_AXIS)....64
12.3 Signal Semantics....65
13 AXI-Stream Rich Interface (TSTRB + AMBA5)....66
13.1 Rich Master Interface (S_AXIS)....66
13.2 Rich Slave Interface (M_AXIS)....68
13.3 TSTRB Semantics....69
13.4 TWAKEUP Semantics....70
13.5 Check-Signal Semantics (IHI 0051B Chapter 5)....71
13.6 Parameter Implications....72
14 Clock and Reset....73
14.1 Clock....73
14.2 Reset....74
14.3 Clocking Rules....75
14.4 Reset Recommendations....76
15 Throughput Characteristics....77
15.1 Aggregate Throughput....77
15.2 Per-Slave Throughput....78
15.3 Head-of-Line Blocking....79
15.4 Throughput Targets....80
15.5 Factors That Reduce Throughput....81
16 Latency Analysis....82

16.1 Cycle Breakdown....82
16.2 Arbitration Delay....83
16.3 Backpressure Latency....84
16.4 Latency Targets....85
16.5 Frequency Target....86
17 Resources....87
17.1 Estimates from the Performance Model....87
17.2 Scaling Laws....88
17.3 Area Optimization Knobs....89
18 System Requirements....90
18.1 Toolchain Requirements....90
18.2 Runtime Requirements....91
18.3 Interface Assumptions....92
18.4 Integration Checklist....93
19 Parameter Configuration....94
19.1 Generated Module Parameters....94
19.2 Generator CLI Options....95
19.3 Example Configurations....96
19.3.1 Base flavor 4×16 @ 64-bit....96
19.3.2 Rich flavor 2×4 @ 32-bit with TSTRB and AMBA5....96
19.3.3 Base flavor without counters....96
20 Verification Strategy....97
20.1 Current Status....97
20.2 Generator Verification....98

20.3 Lint Verification....99
20.4 Planned RTL Verification....100
20.5 Formal Properties....101
21 Synthesis and Implementation....102
21.1 Synthesis Flow....102
21.2 Timing Considerations....103
21.3 Floorplanning Suggestions....104
21.4 Implementation Checklist....105
22 Mesh Hierarchy....106
22.1 Placement Levels....106
22.1.1 Figure 7.1: Delta Mesh Placement Hierarchy....107
22.2 Cell....108
22.3 Tile....109
22.4 Confluence....110
22.5 Fabric....111
22.6 Why This Grouping....112
23 Cell and Flit....113
23.1 Cell Ports....113
23.2 Clock and Reset....114
23.3 Flit Format....115
23.3.1 Head-Flit Header Fields....115
24 Routing and Configuration....117
24.1 Routing Tables....117
24.2 Reset Default....118

24.3 Table Constraints....119
24.4 In-Band Configuration....120
24.5 Broadcast Mode-Switch....121
25 Flow Control and Deadlock....122
25.1 Credit-Based Flow Control....122
25.2 Two Virtual Channels....123
25.3 VC Transition Rules....124
25.4 Torus Wrap and the Dateline Rule....125
25.5 Livelock....126
26 Quality of Service....127
26.1 Three Traffic Classes....127
26.2 Strict Priority With Aging....128
26.3 Non-Interference....129
26.4 Per-Class Routes....130
26.5 Within-Class Fairness....131
27 Multicast....132
27.1 VC0 Only....132
27.2 BRCP-Conformed Paths....133
27.3 Source-Side Replication....134
27.4 Multiple Delivery VCs....135
27.5 No Responses....136
28 Systolic Support and Fabric Modes....137
28.1 Torus Wrap Links....137
28.2 Route Lookahead....138

28.3 Source-Route Mode....139
28.4 Standard Fabric Modes....140
28.4.1 Standard Fabric Modes....140
29 Observability and DFT....142
29.1 Ocelus Counter Suite....142
29.2 DFT Test Mode....143
30 Performance and Verification....144
30.1 Performance Model....144
30.2 Verification Plan....145
31 Deferred Items....146

# List of Figures

Figure 2.1: Delta System Context....44
Figure 3.1: Delta High-Level Block Diagram....50
Figure 7.1: Delta Mesh Placement Hierarchy....107

# List of Tables

Table 1: Document Properties....13
Table 2: Revision History....14
Table 3: Naming Conventions....23
Table 4: Acronyms and Definitions....28
Table 5: Delta Component Context....48
Table 6: Topology Parameter Scaling....52
Table 7: Protocol Support Matrix....62
Table 8: Base Master Interface Signals....63
Table 9: Base Slave Interface Signals....64
Table 10: Rich Master Interface Additional Signals....66
Table 11: Rich Slave Interface Additional Signals....68
Table 12: Rich-Flavor Parameter-to-Port Mapping....72
Table 13: Clock Signal....73
Table 14: Reset Signal....74
Table 15: Throughput Targets....80
Table 16: Latency Cycle Breakdown....82
Table 17: Latency Targets....85
Table 18: Resource Estimates....87
Table 19: Area Optimization Options....89
Table 20: Generated Module Parameters....94
Table 21: Generator Command-Line Options....95
Table 22: Head-flit header fields carried in TUSER on the head beat....115
Table 23: Standard fabric modes selected by broadcast mode-switch....139

# 1 Delta Has Index

**Generated:** 2026-10-14

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 Document Information

## 2.1 Delta Hardware Architecture Specification

*Document Properties*

| Property | Value |
|---|---|
| Document Title | Delta Hardware Architecture Specification |
| Version | 1.0 |
| Date | 2026-10-12 |
| Status | Released |
| Classification | Open Source - MIT License |

## 2.2 Revision History

*Revision History*

| Version | Date | Author | Description |
|---|---|---|---|
| 1.0 | 2026-10-12 | RTL Design Sherpa | Initial HAS/MAS split; documents the 2026-10-12 richness options (TSTRB and AMBA5 TWAKEUP/TPARITY) and flat-only topology. |
| 1.1 | 2026-10-12 | RTL Design Sherpa | AMBA 5 interface parity per IHI 0051B ch. 5 (issue #102): `TPARITY`/`PW_WIDTH`/`--parity-width` replaced by the odd-parity check signals `TDATACHK` … `TWAKEUPCHK` (spec widths, checked at every port, sticky `*_parity_error`), matching `rtl/amba/axis5`. First `dv/` suite. |

## 2.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Delta AXI4-Stream crossbar generator — the map you read before you integrate, not the schematic. It covers:

- System-level architecture and data flow
- AXI4-Stream protocol support (base and rich flavors)
- Interface definitions and requirements
- Performance characteristics
- Integration guidelines

For micro-architecture, FSM designs, and signal-level detail, see the companion **Delta Micro-Architecture Specification (MAS)**.

## 2.4 Intended Audience

- System architects defining streaming interconnect topology
- Hardware engineers integrating Delta into SoC designs
- Verification engineers planning test strategies
- Technical managers evaluating Delta capabilities

## 2.5 Related Documents

- **Delta MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI assistant integration guide
- **QUICK_START.md** - Hands-on generator guide

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 3 Purpose and Scope

## 3.1 What is Delta?

**Delta** is a Python-based AXI4-Stream crossbar generator. You provide the number of masters, the number of slaves, the data width, and a few optional sideband knobs; it hands you back parameterized, synthesizable SystemVerilog. The name follows the water theme used elsewhere in RTL Design Sherpa (like RAPIDS): a river delta is where one channel splits into many branches, analogous to crossbar routing.

## 3.2 The Problem Delta Solves

**A hand-written AXI4-Stream crossbar is error-prone.** Building one means writing per-slave arbitration, packet-atomic grant logic, data multiplexing for TDATA/TVALID/TLAST/TDEST/TID/TUSER, and TREADY backpressure steering — then doing it again every time the master or slave count changes.

**Delta automates all of it:**

- Define the crossbar in a single command line
- Run `bin/delta_generator.py`
- Get production-ready SystemVerilog RTL

## 3.3 Scope

This specification covers:

- High-level architecture and block organization
- AXI4-Stream protocol support, including the optional base + rich signal sets
- Interface requirements and signal conventions
- Performance characteristics and resource estimates
- Integration guidelines and verification status

## 3.4 Out of Scope

These live in the companion MAS or are deliberate non-goals:

- Detailed FSM state diagrams and transitions (MAS)
- Signal-level timing diagrams (MAS)
- Internal pipeline structures (MAS)
- AXI4/AXI4-Lite/APB protocol conversion (deliberate non-goal; see Bridge or APB crossbar siblings)
- Dynamic reconfiguration or advanced routing algorithms

## 3.5 Non-Goal: Memory-Mapped Protocol Conversion

Delta is **AXI4-Stream only**. It does not convert to or from AXI4, AXI4-Lite, or APB. If you need address-mapped fabrics, use the sibling `bridge` or `lattice-apbx-xbar` components.

# 4 Document Conventions

## 4.1 Naming Conventions

*Naming Conventions*

| Convention | Meaning | Example |
|---|---|---|
| `s_axis_*` | Signals on the master side of the crossbar (inputs) | `s_axis_tdata[m]` |
| `m_axis_*` | Signals on the slave side of the crossbar (outputs) | `m_axis_tready[s]` |
| `[NUM_MASTERS]` | Unpacked array dimension, one entry per master | `s_axis_tvalid [NUM_MASTERS]` |
| `[NUM_SLAVES]` | Unpacked array dimension, one entry per slave | `m_axis_tvalid [NUM_SLAVES]` |
| `m` | Master index | `for (int m = 0; m < NUM_MASTERS; m++)` |
| `s` | Slave index | `for (int s = 0; s < NUM_SLAVES; s++)` |

## 4.2 Signal Direction Conventions

All directions are stated from the crossbar's point of view:

- `s_axis_*` signals are driven by upstream masters and received by Delta.
- `m_axis_*` signals are driven by Delta and received by downstream slaves.
- `aclk` and `aresetn` are driven by the system.

## 4.3 Numbering Conventions

- Masters are numbered `0` to `NUM_MASTERS-1`.
- Slaves are numbered `0` to `NUM_SLAVES-1`.
- `TDEST` is interpreted as a slave index; legal values are `0` to `NUM_SLAVES-1`.

## 4.4 Typographic Conventions

- `code font` is used for signal names, parameters, and file paths.
- **Bold** is used for emphasis and defined terms.
- Tables use the RTL Design Sherpa corporate document style.

## 4.5 Versioning Conventions

This document uses semantic-style major.minor revision numbers. The initial release is 1.0.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 5 Definitions and Acronyms

## 5.1 Acronyms

*Acronyms and Definitions*

| Acronym | Definition |
|---|---|
| AXIS | AXI4-Stream (ARM AMBA streaming protocol) |
| AMBA5 | ARM AMBA 5 protocol extensions, including TWAKEUP and the T*CHK interface check signals |
| TDEST | Transaction destination (slave ID in AXI4-Stream) |
| TID | Transaction ID (master identifier in AXI4-Stream) |
| TUSER | User-defined sideband in AXI4-Stream |
| TSTRB | Byte strobe / byte qualifier in AXI4-Stream |
| TWAKEUP | AMBA 5 wakeup hint signal |
| T*CHK | AXI5-Stream interface check signals (`TDATACHK`, `TVALIDCHK`, …), odd parity, IHI 0051B chapter 5 |
| RR | Round-robin arbitration |
| LUT | Look-up table (FPGA logic resource) |
| FF | Flip-flop (FPGA register resource) |

## 5.2 Key Terms

**Base flavor**
The default generated crossbar with the seven-signal set: `tdata`, `tvalid`, `tready`, `tlast`, `tdest`, `tid`, `tuser`.

**Rich flavor**
A generated crossbar with optional additional signals enabled: `tstrb` and/or AMBA 5 `twakeup` with the `t*chk` check signals.

**Packet atomicity**
The property that once a master is granted access to a slave, it retains that grant until the current packet's `TLAST` beat has been transferred. No other master may interleave beats on that slave during the packet.

**Request matrix**
A per-slave collection of master request bits. `request_matrix[s][m]` is true when master `m` has a valid transaction targeting slave `s`.

**Grant matrix**
A per-slave one-hot (or zero-hot) collection of master grant bits. `grant_matrix[s][m]` is true when master `m` currently owns slave `s`.

**TDEST decode**
The combinational translation of `s_axis_tdest[m]` into a slave index in the request matrix. Delta does not use an address map.

**TWAKEUP steering**
The AMBA 5 sideband path that forwards a master's `s_axis_twakeup` hint to the slave selected by `s_axis_tdest`. It is an OR-per-slave and may assert before `TVALID`.

# 6 Use Cases

## 6.1 Primary: RISC Cores + DSP Array

**Scenario:** Four small RISC-V cores route compute tasks to sixteen DSP accelerators.

**Configuration:**

- 4 masters (RISC cores)
- 16 slaves (DSP arrays)
- 64-bit data width
- Flat topology

**Benefits:**

- Each RISC core can target any DSP (full flexibility).
- Per-slave round-robin arbitration prevents starvation.
- Two-cycle latency keeps task dispatch responsive.
- All sixteen DSPs can operate concurrently when traffic targets different slaves.

## 6.2 Secondary: Sensor/Peripheral Fabric

**Scenario:** Multiple sensor streams feed a smaller set of processing blocks.

**Configuration:**

- 2-8 masters (sensor front-ends)
- 2-4 slaves (aggregation or filter blocks)
- 32-bit data width with optional `TSTRB` byte qualifiers

**Benefits:**

- Direct TDEST routing removes the need for an address decoder.
- Packet atomicity preserves frame boundaries from each sensor.
- Optional `TSTRB` supports sparse byte-valid data from byte-packed sensors.

## 6.3 Tertiary: AMBA 5-Aware Streaming Interconnect

**Scenario:** A low-power subsystem needs wakeup hints and AXI5-Stream interface parity protection.

**Configuration:**

- Masters and slaves that implement AMBA 5 AXI-Stream
- `--amba5` enabled

**Benefits:**

- `TWAKEUP` lets a master wake a target slave before presenting data.
- The IHI 0051B chapter 5 check signals (`TDATACHK` and the rest, odd parity) are carried across the fabric and checked at each crossbar port. A sticky per-port flag reports a fault.
- Generated ports port-match the `rtl/amba/axis5/*` interface conventions.

## 6.4 Educational Demonstration

**Scenario:** Teach interconnect design trade-offs and code generation.

**Configuration:**

- Generate flat crossbars of varying sizes into `rtl_test/`.
- Compare latency, throughput, and resource estimates from `delta_performance_model.py`.

**Benefits:**

- Hands-on learning with working RTL.
- Clear performance comparisons.
- Demonstrates specification-driven design and Python-based RTL generation.

## 6.5 Mesh-Mode: Monitored Manycore Fabric

**Scenario:** A 3-D manycore fabric of SERV cores and CGRA compute cells exchanges stream traffic while ocelus observers collect statistics and rapids DMA feeds the boundary.

**Configuration:**

- `--topology mesh` with fabric dimensions `cells-x`, `cells-y`, `cells-z`.
- One `delta_mesh_cell` at every fabric coordinate; rapids attaches at boundary cells' local ports.
- ocelus observers tap cell ports for flit/packet/stall/latency counters.
- Cell contents (SERV, CGRA, ocelus) are out of scope — delta sees only the AXI4-Stream at the local port.

  **PLANNED** — The mesh cell RTL is not yet implemented. The design described here is per the owner-approved delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`). The flat crossbar remains the implemented default.

**Benefits:**

- Table-routed 3-D mesh scales without the all-to-all wiring of a flat crossbar.
- Per-cell routing tables and broadcast mode-switches let the same physical fabric run different communication patterns.
- Built-in observability gives software-visible performance data from the fabric.

# 7 Key Features

## 7.1 Flat M×N Crossbar

Delta generates a single self-contained SystemVerilog module implementing a full M×N crossbar. Every master can reach every slave; each slave has its own round-robin arbiter.

## 7.2 TDEST-Based Routing

There is no address map. The target slave is taken directly from `s_axis_tdest`. This makes request decode simpler than an address-based crossbar and removes configuration risk.

## 7.3 Packet-Atomic Round-Robin Arbitration

Each slave arbiter grants one master at a time and holds that grant until the current packet's TLAST beat is accepted. This guarantees:

- No packet interleaving on a slave.
- Fair bandwidth allocation among contending masters.
- Streaming packet boundaries are preserved end-to-end.

## 7.4 Combinational Data Mux with Registered Grant

The data path from the granted master to the selected slave is combinational. The grant decision is registered on `aclk`, giving a predictable two-cycle latency while keeping the critical path short.

## 7.5 Optional Richness (Default Off)

Two command-line options add sidebands without changing the default output:

- `--strb-width N` adds TSTRB byte qualifiers (port-matches `rtl/amba/axis4`).
- `--amba5` adds TWAKEUP and the IHI 0051B chapter 5 odd-parity check signals (TDATACHK ... TWAKEUPCHK), with sticky per-port error flags. The ports match `rtl/amba/axis5`.

When neither option is used, the generated module is byte-identical to pre-2026-10-12 output.

## 7.6 Optional Packet Counters

Per-master and per-slave 32-bit packet counters count completed packets (`TVALID && TREADY && TLAST`). Disable them with `--no-counters` to save area.

## 7.7 Formal Assertions

When `FORMAL` is defined, the generated RTL includes assertions that check:

- One-hot (or zero-hot) grants per slave.
- `TDEST` is within range when `s_axis_tvalid` is asserted.

## 7.8 Code Generation

The generator is a Python script (`bin/delta_generator.py`) that reuses the unified `Module` framework. It produces lint-clean SystemVerilog and is the single source of truth for all checked-in RTL variants.

## 7.9 Mesh-Mode Features

**PLANNED** — Mesh mode is a planned generator option, not yet implemented RTL. The design below is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`). The flat TDEST crossbar remains the default and is untouched.

- **3-D table-routed mesh of identical cells** — one `delta_mesh_cell` per fabric coordinate, connected to ±X/±Y/±Z neighbors plus one local port.
- **Placement hierarchy** — cell → tile (`2×2×1` = 4 cells) → confluence (`4×4×1` = 16 cells) → fabric (`4×4×K` confluences stacked in Z). The hierarchy names regions and bounds table size; there is no wrapper RTL between tile or confluence boundaries.
- **2 VCs with escape-channel deadlock freedom** — VC0 carries the connected, acyclic escape subrelation; VC1 carries minimal-or-budgeted adaptive routes. VC1 may drain into VC0, but VC0 may never enter VC1.
- **3 QoS classes** — config, latency, and bulk traffic get separate VCs and injection buffers, with strict priority, programmable aging, and a bulk minimum reservation.
- **In-band config with broadcast mode-switch** — per-cell RAM tables load through the data path; a broadcast mode-switch message switches all cells to one of 8 standard fabric modes atomically after a fabric-wide ack barrier.
- **8 standard fabric modes** — mesh, systolic-1D, wavefront-2D, stencil-3D, broadcast, convergecast/reduce, all-to-all/transpose, and pipeline+rapids-stream.
- **Multicast** — VC0-only, BRCP-conformed paths with source-side replication, multiple delivery VCs, and no responses.
- **Systolic/torus package** — per-dimension torus wrap (`--torus=x,y,z`), route lookahead, and source-route mode for fixed-latency systolic schedules.

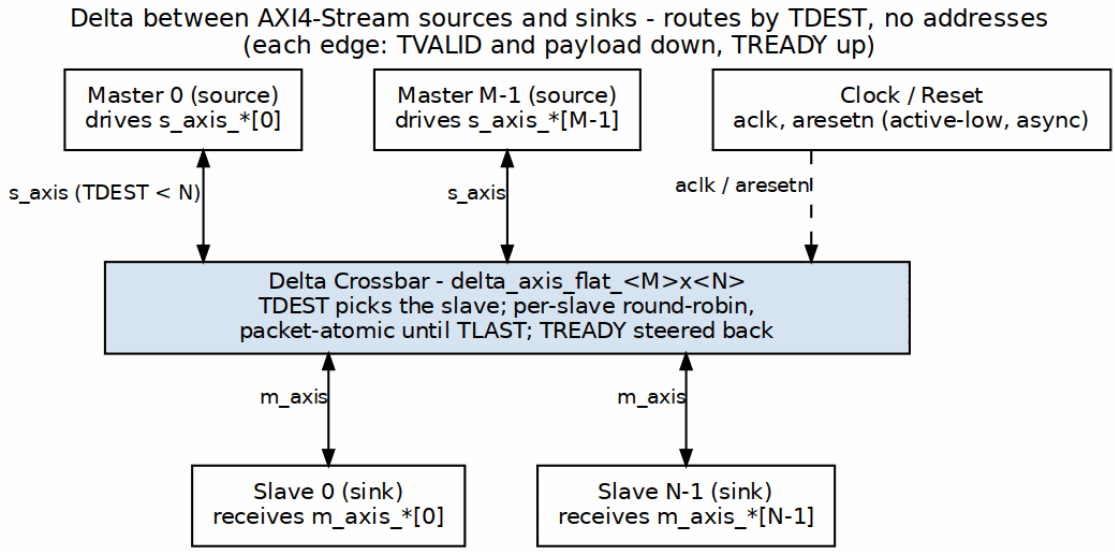
# 8 System Context

## 8.1 Where Delta Fits

Delta sits between upstream AXI4-Stream masters and downstream AXI4-Stream slaves. It does not interpret addresses, bursts, or memory attributes; it only routes streams based on `TDEST`.

A typical Delta deployment looks like:

### 8.1.1 Figure 2.1: Delta System Context




*Delta System Context*

Multiple masters and multiple slaves attach in parallel. Each master drives one index of the `s_axis_*` arrays; each slave receives one index of the `m_axis_*` arrays.

## 8.2 Upstream Requirements

- Masters must drive valid AXI4-Stream signals.
- `s_axis_tdest[m]` must be a valid slave index (`0` to `NUM_SLAVES-1`) when `s_axis_tvalid[m]` is asserted.
- Masters must respect `s_axis_tready[m]` backpressure.

## 8.3 Downstream Requirements

- Slaves must accept valid AXI4-Stream signals.
- Slaves drive `m_axis_tready[s]` to throttle traffic.

## 8.4 Clocking and Reset

Delta uses a single clock (`aclk`) and an active-low asynchronous reset (`aresetn`). All registered state is reset; combinational decode and mux logic is not registered.

## 8.5 Relationship to Other Components

*Delta Component Context*

| Component | Role | When to Use |
|---|---|---|
| Delta | AXI4-Stream crossbar generator | Streaming fabrics, DSP arrays, sensors |
| Bridge | Multi-protocol crossbar generator | Memory-mapped AXI4/AXI4-Lite/APB systems |
| APB Crossbar | APB-only crossbar | Low-bandwidth register-mapped peripherals |
| RAPIDS | DMA subsystem | Descriptor-based memory transfers |

Delta is intentionally narrow in scope: it does one thing (streaming crossbars) and does it without protocol conversion.

## 8.6 Mesh-Mode Context

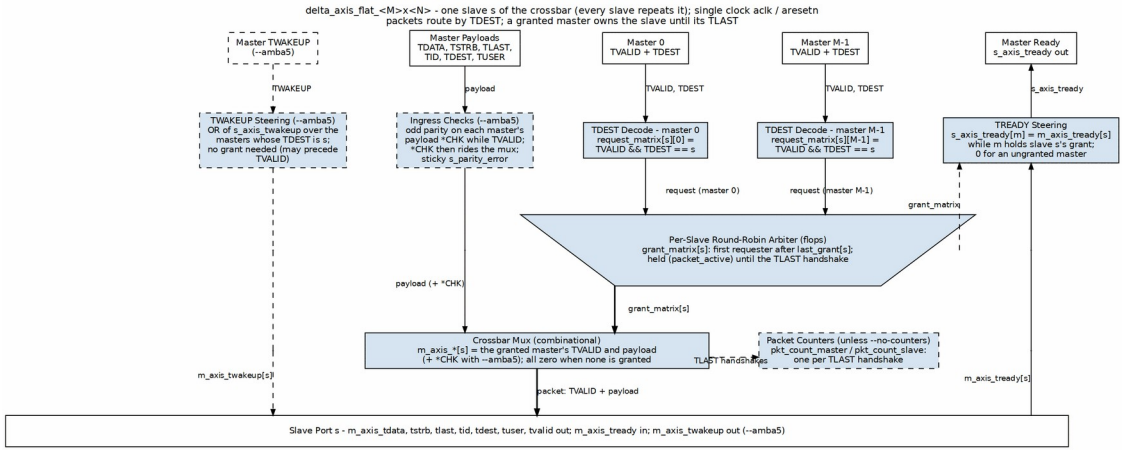
Mesh mode extends the same `bin/delta_generator.py` with a `--topology mesh` option. Instead of a flat M×N crossbar it emits a 3-D grid of `delta_mesh_cell` instances with rapids DMA at the boundary and ocelus observers on the links. Cell contents (SERV, CGRA, observers) are outside delta's scope; delta only carries the AXI4-Stream traffic between them.

> **PLANNED** — Mesh mode RTL is not yet implemented; the design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`). The flat crossbar shown in Figure 2.1 is the current implementation.

# 9 Block Diagram

## 9.1 Delta Architecture Overview

### 9.1.1 Figure 3.1: Delta High-Level Block Diagram




*Delta HAS Block Diagram*

The diagram follows one slave `s`; every slave repeats it. Each master's TVALID and TDEST are decoded into a request for `s` (center), the per-slave round-robin arbiter (the trapezium) grants one master and holds the grant until that packet's TLAST, and the crossbar mux forwards the granted master's TVALID and payload, which arrive on their own lane at the left. TREADY returns up the right lane to the granted master only. The dashed blocks are build options: `--amba5` adds TWAKEUP steering and the `*CHK` parity checks, and the packet counters are built unless `--no-counters`.

## 9.2 Functional Blocks

### 9.2.1 TDEST Decode

Receives `s_axis_tvalid` and `s_axis_tdest` from every master and produces the per-slave `request_matrix`. A request bit is set when a master has a valid transaction whose TDEST selects that slave.

### 9.2.2 Per-Slave Round-Robin Arbiter

One arbiter per slave. When the slave is idle, it picks the next requesting master in round-robin order. Once a master is granted, the grant is held until TLAST is accepted. This block is the scheduler analog in the house style and is drawn as an inverted trapezium.

### 9.2.3 Crossbar Mux

A bank of per-slave multiplexers. Each slave mux selects the payload and control signals from the master that currently holds the grant for that slave.

### 9.2.4 TREADY Steering

Steers each slave's `m_axis_tready` back to the single master that currently owns the grant for that slave. Ungranted masters see `s_axis_tready == 0`.

### 9.2.5 TWAKEUP Steering (AMBA5, Optional)

When `--amba5` is enabled, this block ORs each master's `s_axis_twakeup` to the slave indicated by `s_axis_tdest`. Because TWAKEUP is a hint, it does not depend on the grant and may assert before TVALID.

### 9.2.6 Interface Checks (AMBA5, Optional)

With `--amba5`, each master's payload check bits (TDATACHK … TUSERCHK) are checked for odd parity while TVALID is high and then carried through the mux with the beat; TVALIDCHK, TWAKEUPCHK and TREADYCHK are generated by the side that drives the signal and checked by the other (`dataint_parity`). A failed check sets the port's sticky `s_parity_error` / `m_parity_error`.

### 9.2.7 Packet Counters (Optional)

32-bit counters per master and per slave that increment on every completed packet (`TVALID && TREADY && TLAST`). Removed from the build with `--no-counters`.

## 9.3 Scalability

*Topology Parameter Scaling*

| Parameter | Typical Range | Impact |
|---|---|---|
| NUM_MASTERS | 1-32 | Linear arbitration and mux growth |
| NUM_SLAVES | 1-256 | Linear decode and grant-matrix growth |
| DATA_WIDTH | 8-1024 | Linear data-path width |
| DEST_WIDTH | auto (`clog2(NUM_SLAVES)`) | Sets TDEST width |
| ID_WIDTH | auto (`clog2(NUM_MASTERS)`) | Sets TID width |

### 9.3.1 Resource Scaling

- **Logic:** O(NUM_MASTERS × NUM_SLAVES) for the grant and request matrices.
- **Data path:** O(NUM_SLAVES × DATA_WIDTH) for the output muxes.
- **Memory:** No BRAM; all state is flip-flops.

# 10 Data Flow

## 10.1 Request Flow

1. A master asserts `s_axis_tvalid[m]` and places the target slave index on `s_axis_tdest[m]`.
2. The TDEST decode block sets `request_matrix[s][m]` for the selected slave.
3. If the slave is not already serving another packet, the per-slave round-robin arbiter grants one requesting master.
4. The grant is registered and held until the packet ends on `TLAST`.

## 10.2 Payload Flow

1. While a grant is active, the crossbar mux selects the granted master's payload.
2. `m_axis_tdata[s]`, `m_axis_tlast[s]`, `m_axis_tdest[s]`, `m_axis_tid[s]`, and `m_axis_tuser[s]` are driven from the granted master.
3. If `TSTRB` is enabled, `m_axis_tstrb[s]` is also forwarded.
4. If `--amba5` is enabled, the payload check bits (`m_axis_tdatachk[s]` through `m_axis_tuserchk[s]`) are forwarded with the beat. They were already checked at the master port, where a failure sets `s_parity_error[m]`.

## 10.3 Backpressure Flow

1. Each downstream slave drives `m_axis_tready[s]`.
2. The TREADY steering block routes the granted slave's ready back to the owning master.
3. Only the granted master sees `s_axis_tready[m] == 1`; all other masters are stalled.

## 10.4 Wakeup Flow (AMBA5)

1. A master may assert `s_axis_twakeup[m]` before asserting `s_axis_tvalid[m]`.
2. The TWAKEUP steering block decodes `s_axis_tdest[m]` and ORs the hint to the matching slave.
3. `m_axis_twakeup[s]` may assert with no active grant.

## 10.5 Packet Boundaries

TLAST marks the final beat of a packet. The arbiter releases the grant on the cycle in which `m_axis_tvalid[s] && m_axis_tready[s] && m_axis_tlast[s]` is true. The next arbitration decision can occur on the following cycle.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 11 Protocol Support

## 11.1 AXI4-Stream Base Flavor

The default generated crossbar implements the standard AXI4-Stream signal set:

- `TDATA` — data payload
- `TVALID` / `TREADY` — handshake
- `TLAST` — packet boundary
- `TDEST` — destination slave index
- `TID` — transaction/master ID (pass-through)
- `TUSER` — user sideband (pass-through)

This is the byte-identical default; no sideband options are enabled unless requested.

## 11.2 Optional TSTRB Byte Qualifiers

The `--strb-width N` option adds TSTRB to every master and slave interface. TSTRB is treated as a pass-through sideband: the mux forwards the granted master's TSTRB to the selected slave. Delta does not interpret or modify strobe bits.

## 11.3 Optional AMBA 5 Sidebands

The `--amba5` option adds:

- `TWAKEUP`: a wakeup hint, decoded from `TDEST` and ORed per slave.
- The AXI5-Stream interface-protection signals of IHI 0051B chapter 5 (`Check_Type = Odd_Parity_Byte_All`): `TDATACHK`, `TSTRBCHK` (with `TSTRB`), `TLASTCHK`, `TIDCHK`, `TDESTCHK`, `TUSERCHK`, `TVALIDCHK`, `TREADYCHK` and `TWAKEUPCHK`, all odd parity. The spec fixes the widths at one check bit per 8 bits.
- Sticky `s_parity_error` / `m_parity_error` flags per port.

The crossbar checks the payload check bits on ingress and carries them to the granted slave unchanged. It generates the control check bits for the signals it drives and checks the ones it receives. The generated ports match the `rtl/amba/axis5/*` modules. See Chapter 4, AXI-Stream Rich Interface, for the full table.

## 11.4 What Delta Does Not Support

Delta is intentionally AXI4-Stream only. It does **not** support:

- AXI4 memory-mapped transactions (AW/AR/W/B/R channels).
- AXI4-Lite register-mapped access.
- APB transfers.
- Burst lengths, address phases, or memory attributes.
- Dynamic reconfiguration of the crossbar dimensions.

For memory-mapped fabrics, use the sibling `bridge` or `lattice-apbx-xbar` components.

## 11.5 Protocol Compliance Summary

*Protocol Support Matrix*

| Feature | Base Flavor | Rich Flavor | Notes |
|---|---|---|---|
| TVALID/ TREADY handshake | Yes | Yes | Standard AXI4-Stream |
| TLAST packet boundary | Yes | Yes | Grant release on TLAST |
| TDEST routing | Yes | Yes | Direct slave index |
| TID/TUSER pass-through | Yes | Yes | Unmodified |
| TSTRB | No | Yes | `--strb-width N` |
| TWAKEUP | No | Yes | `--amba5` |
| T*CHK interface parity (IHI 0051B ch. 5, odd) | No | Yes | `--amba5` |

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 12 AXI-Stream Base Interface

## 12.1 Master Interface (S_AXIS)

Each master port is indexed by `m` in the range `0` to `NUM_MASTERS-1`.

*Base Master Interface Signals*

| Signal | Width | Direction | Description |
|---|---|---|---|
| `s_axis_tdata[m]` | `DATA_WIDTH` | Input | Data payload. |
| `s_axis_tvalid[m]` | 1 | Input | Valid indicator. When high, `s_axis_tdest[m]` selects the target slave. |
| `s_axis_tready[m]` | 1 | Output | Ready / backpressure. Driven from the granted slave's `m_axis_tready`. |
| `s_axis_tlast[m]` | 1 | Input | Packet boundary. The final beat of a packet. |
| `s_axis_tdest[m]` | `DEST_WIDTH` | Input | Target slave index. Must be `< NUM_SLAVES` when `tvalid` is asserted. |
| `s_axis_tid[m]` | `ID_WIDTH` | Input | Transaction ID. Pass-through to `m_axis_tid`. |
| `s_axis_tuser[m]` | `USER_WIDTH` | Input | User sideband. Pass-through to `m_axis_tuser`. |

## 12.2 Slave Interface (M_AXIS)

Each slave port is indexed by `s` in the range `0` to `NUM_SLAVES-1`.

*Base Slave Interface Signals*

| Signal | Width | Direction | Description |
|---|---|---|---|
| `m_axis_tdata[s]` | `DATA_WIDTH` | Output | Data payload from the granted master. |
| `m_axis_tvalid[s]` | 1 | Output | Valid indicator. Asserted when a grant is active and the selected master is valid. |
| `m_axis_tready[s]` | 1 | Input | Ready / backpressure from the downstream slave. |
| `m_axis_tlast[s]` | 1 | Output | Packet boundary. Forwarded from the granted master. |
| `m_axis_tdest[s]` | `DEST_WIDTH` | Output | Target slave index. Pass-through from the granted master. |
| `m_axis_tid[s]` | `ID_WIDTH` | Output | Transaction ID. Pass-through from the granted master. |
| `m_axis_tuser[s]` | `USER_WIDTH` | Output | User sideband. Pass-through from the granted master. |

## 12.3 Signal Semantics

- `TVALID` may be asserted without `TREADY`; the transfer completes only when both are high.
- `TREADY` for an ungranted master is always low.
- `TDEST` must be stable while `TVALID` is asserted and the master is waiting for grant or transfer.
- `TID` and `TUSER` are pass-through and must be stable on every valid beat.

# 13 AXI-Stream Rich Interface (TSTRB + AMBA5)

## 13.1 Rich Master Interface (S_AXIS)

When `--strb-width N` and/or `--amba5` are used, the master interface adds the following signals in addition to the base set. The check signals are IHI 0051B chapter 5 (`Check_Type = Odd_Parity_Byte_All`). Names and widths match `rtl/amba/axis5/*`.

*Rich Master Interface Additional Signals*

| Signal | Width | Direction | Presence | Description |
|---|---|---|---|---|
| `s_axis_tstrb[m]` | `STRB_WIDTH` | Input | `--strb-width N` | Byte qualifier / strobe. Pass-through to `m_axis_tstrb`. |
| `s_axis_twakeup[m]` | 1 | Input | `--amba5` | Wakeup hint. May assert before `TVALID`. |
| `s_axis_tdatachk[m]` | `TDATACHK_W = DATA_WIDTH/8` | Input | `--amba5` | Odd parity per `TDATA` byte. Checked, then carried. |
| `s_axis_tstrbchk[m]` | `TSTRBCHK_W = ceil(STRB_WIDTH/8)` | Input | `--amba5` and `--strb-width N` | Odd parity over `TSTRB`. Checked, then carried. |
| `s_axis_tlastchk[m]` | 1 | Input | `--amba5` | ~`TLAST`. Checked, then carried. |
| `s_axis_tidchk[m]` | `TIDCHK_W = ceil(ID_WIDTH/8)` | Input | `--amba5` | Odd parity over `TID`. Checked, then carried. |
| `s_axis_tdestchk[m]` | `TDESTCHK_W = ceil(DEST_WIDTH/8)` | Input | `--amba5` | Odd parity over `TDEST`. Checked, then carried. |
| `s_axis_tuserchk[m]` | `TUSERCHK_W =` | Input | `--amba5` | Odd parity over `TUSER`. Checked, |

| Signal | Width | Direction | Presence | Description |
|---|---|---|---|---|
| | ceil(USER_WIDTH/8) | | | then carried. |
| s_axis_tvalidchk[m] | 1 | Input | --amba5 | ~TVALID. Checked every cycle out of reset. |
| s_axis_treadychk[m] | 1 | Output | --amba5 | ~TREADY, generated by the crossbar. |
| s_axis_twakeupchk[m] | 1 | Input | --amba5 | ~TWAKEUP. Checked every cycle out of reset. |
| s_parity_error[m] | 1 | Output | --amba5 | Sticky. Set by any check failure on master port m. Cleared by aresetn. |

## 13.2 Rich Slave Interface (M_AXIS)

*Rich Slave Interface Additional Signals*

| Signal | Width | Direction | Presence | Description |
|---|---|---|---|---|
| m_axis_tstrb[s] | STRB_WIDTH | Output | --strb-width N | Byte qualifier from the granted master. |
| m_axis_twakeup[s] | 1 | Output | --amba5 | Wakeup hint. OR of all masters whose TDEST selects slave s. |
| m_axis_tdatachk[s], m_axis_tstrbchk[s], m_axis_tlastchk[s], m_axis_tidchk[s], m_axis_tdestchk[s], m_axis_tuserchk[s] | as on the master side | Output | --amba5 (tstrbchk also needs --strb-width N) | Payload check bits of the granted master, carried unchanged. All ones while no master is granted. |
| m_axis_tvalidchk[s] | 1 | Output | --amba5 | ~TVALID, generated by the crossbar. |
| m_axis_treadychk[s] | 1 | Input | --amba5 | ~TREADY. Checked every cycle out of reset. |
| m_axis_twakeupchk[s] | 1 | Output | --amba5 | ~TWAKEUP, generated by the crossbar. |
| m_parity_error[s] | 1 | Output | --amba5 | Sticky. Set by a bad TREADYCHK on slave port s. Cleared by aresetn. |

## 13.3 TSTRB Semantics

- `TSTRB` is treated as an opaque byte-qualifier sideband.
- The crossbar forwards the granted master's `TSTRB` to the selected slave.
- Delta does not gate `TVALID` based on `TSTRB` or modify strobe bits.

## 13.4 TWAKEUP Semantics

TWAKEUP is an AMBA 5 hint, not a handshake signal:

- A master may assert `s_axis_twakeup[m]` before `s_axis_tvalid[m]` to wake the target slave.
- The crossbar decodes `s_axis_tdest[m]` and ORs the hint to the indicated slave.
- `m_axis_twakeup[s]` may be high while `m_axis_tvalid[s]` is low.
- No grant is required for TWAKEUP steering.

## 13.5 Check-Signal Semantics (IHI 0051B Chapter 5)

- Parity is odd. A check bit and the bits it covers hold an odd number of ones. Bit `[n]` covers bits `[8n+7:8n]`, and the top bit covers a short remainder (section 5.3).
- Payload checks (`TDATACHK` through `TUSERCHK`) are checked only while `TVALID` is high. `TVALIDCHK`, `TREADYCHK` and `TWAKEUPCHK` are checked on every cycle out of reset.
- Payload check bits are carried end to end with the beat, never regenerated. A bad bit at a master port stays bad at the slave port.
- A failed check sets the port's sticky error flag. It does not stall, drop or alter the beat.
- Every check bit is computed or checked by a `rtl/common/dataint_parity` instance. The compile closure of a generated `--amba5` crossbar includes `rtl/common/filelists/dataint_parity.f`.

## 13.6 Parameter Implications

*Rich-Flavor Parameter-to-Port Mapping*

| Option | Generated Parameter | Port Effect |
|---|---|---|
| `--strb-width N` | `STRB_WIDTH = N` | Adds `s_axis_tstrb` and `m_axis_tstrb` arrays. |
| `--amba5` | `TDATACHK_W`, `TIDCHK_W`, `TDESTCHK_W`, `TUSERCHK_W` (plus `TSTRBCHK_W` with `--strb-width`), each derived from its payload width | Adds `TWAKEUP`, the nine check signals and the two `*_parity_error` arrays. Requires a whole-byte `--data-width`. |

# 14 Clock and Reset

## 14.1 Clock

*Clock Signal*

| Signal | Width | Direction | Description |
|---|---|---|---|
| `aclk` | 1 | Input | Positive-edge clock for all registered state. |

All registered logic (grants, `last_grant`, packet-active flags, counters) is clocked on the rising edge of `aclk`. Combinational decode and mux logic is not registered.

## 14.2 Reset

*Reset Signal*

| Signal | Width | Direction | Description |
|---|---|---|---|
| `aresetn` | 1 | Input | Active-low asynchronous reset. |

`aresetn` resets all registered state:

- `grant_matrix` to zero.
- `last_grant` to zero.
- `packet_active` to zero.
- Packet counters to zero.

The reset assertion is asynchronous; de-assertion is synchronous to `aclk` in the checked-in RTL through the `ALWAYS_FF_RST` macro.

## 14.3 Clocking Rules

- All masters and slaves share the same `aclk`.
- Delta does not contain clock-domain crossing logic.
- Inputs are sampled on the rising edge of `aclk`; outputs are driven combinationally or from registers.

## 14.4 Reset Recommendations

- Assert `aresetn` for at least several clock cycles after power and clock stability.
- Do not rely on Delta's outputs during reset; combinational mux outputs may be zero or undefined during reset.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 15 Throughput Characteristics

## 15.1 Aggregate Throughput

Because each slave has its own arbiter and mux, aggregate throughput scales with the number of slaves. In the best case — every master targets a different slave and every slave is ready — the crossbar can accept one beat per master per cycle.

## 15.2 Per-Slave Throughput

When multiple masters contend for the same slave, the round-robin arbiter gives each contending master an equal share of the slave's bandwidth. With K contending masters, each sees approximately 1/K of the slave's available bandwidth.

## 15.3 Head-of-Line Blocking

A master blocked on one slave does not block traffic to other slaves. Each slave arbiter is independent. However, a master that targets a heavily contended slave will wait until its round-robin turn and until the current packet completes.

## 15.4 Throughput Targets

*Throughput Targets*

| Metric | Target | Notes |
|---|---|---|
| Per-slave throughput | Up to 1 transfer/cycle | When one master continuously feeds one ready slave. |
| Aggregate throughput | Up to NUM_MASTERS transfers/cycle | When all masters target different ready slaves. |
| Mixed-traffic throughput | ≥ 0.7 transfers/cycle per active master | Under moderate contention in the performance model. |

## 15.5 Factors That Reduce Throughput

- Contention for the same slave.
- Downstream backpressure (`m_axis_tready[s] == 0`).
- Multi-beat packets that lock the slave until `TLAST`.
- Arbitration overhead of one cycle when switching masters after `TLAST`.



---

# 16 Latency Analysis

## 16.1 Cycle Breakdown

The generated flat crossbar has a fixed two-cycle latency from master input to slave output:

*Latency Cycle Breakdown*

| Cycle | Action |
|---|---|
| 0 | Master asserts `s_axis_tvalid` and `s_axis_tdest`. |
| 1 | TDEST decode produces `request_matrix`; arbiter registers the grant. |
| 2 | Crossbar mux drives `m_axis_tdata`, `m_axis_tvalid`, etc. to the slave. |

## 16.2 Arbitration Delay

If the target slave is idle when the request arrives, the grant is registered on the next rising edge of `aclk`. If the slave is busy serving another packet, the new request waits until the current packet's `TLAST` is accepted.

## 16.3 Backpressure Latency

`m_axis_tready[s]` propagates combinationally through the TREADY steering logic to `s_axis_tready[m]`. There is no additional registered delay on the backpressure path.

## 16.4 Latency Targets

*Latency Targets*

| Configuration | Latency | Notes |
|---|---|---|
| Flat crossbar, idle slave | 2 cycles | Arbitration + registered grant. |
| Flat crossbar, contended slave | 2 + wait cycles | Wait depends on current packet length. |

## 16.5 Frequency Target

The flat crossbar is intended to run at **≥ 300 MHz** on Xilinx UltraScale+ devices for typical configurations (e.g., 4×16 @ 64-bit). Actual Fmax depends on device, tool settings, and floorplanning.

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 17 Resources

## 17.1 Estimates from the Performance Model

The `bin/delta_performance_model.py` script provides empirical resource estimates based on the flat topology. Values below are planning estimates, not synthesis results.

*Resource Estimates*

| Configuration | LUTs | FFs | Notes |
|---|---|---|---|
| 4×16 @ 64-bit | ~1,536 | ~1,536 | Base flavor, with counters. |
| 2×4 @ 32-bit | ~200 | ~150 | Small test configuration. |
| 4×16 @ 64-bit, no counters | ~1,400 | ~1,200 | `--no-counters` saves FF area. |

## 17.2 Scaling Laws

- **LUTs:** Grow roughly as `NUM_MASTERS × NUM_SLAVES` for arbitration plus `NUM_SLAVES × DATA_WIDTH` for the data mux.
- **FFs:** Grow roughly as `NUM_MASTERS × NUM_SLAVES` for the grant matrix and `last_grant` storage, plus `NUM_MASTERS + NUM_SLAVES` for packet counters when enabled.
- **BRAM:** None. Delta is a pure-logic implementation.

## 17.3 Area Optimization Knobs

*Area Optimization Options*

| Knob | Effect |
|---|---|
| `--no-counters` | Removes `pkt_count_master` and `pkt_count_slave` arrays. |
| Smaller `NUM_MASTERS`/`NUM_SLAVES` | Linear reduction in control logic. |
| Smaller `DATA_WIDTH` | Linear reduction in data mux area. |
| Disable richness options | Removes `TSTRB` muxing, and with `--amba5` off removes the `TWAKEUP` decode, the check-signal mux and the per-port `dataint_parity` checkers. |

# 18 System Requirements

## 18.1 Toolchain Requirements

- **Synthesis:** Any SystemVerilog-2017-capable synthesis tool (Vivado, Yosys, Genus, etc.).
- **Simulation:** Any SystemVerilog simulator; cocotb is the planned verification framework.
- **Lint:** Verilator (`--lint-only`) is used for quick sanity checks.

## 18.2 Runtime Requirements

- Single clock domain: all `s_axis_*` and `m_axis_*` interfaces use `aclk`.
- Active-low asynchronous reset: `aresetn` must be asserted at power-up.
- All upstream masters and downstream slaves must implement the AXI4-Stream handshake.

## 18.3 Interface Assumptions

- Masters drive valid `TDEST` values (`0` to `NUM_SLAVES-1`).
- Slaves drive `m_axis_tready` independently.
- No master changes `TDEST` while waiting for a grant.
- `TID` and `TUSER` widths match the generated parameters.

## 18.4 Integration Checklist

- ☐ Confirm `DATA_WIDTH`, `NUM_MASTERS`, and `NUM_SLAVES` match the system.
- ☐ Confirm `DEST_WIDTH` and `ID_WIDTH` are wide enough for the generated module.
- ☐ Decide whether `TSTRB` or AMBA 5 sidebands are needed.
- ☐ Decide whether packet counters are needed (default on; use `--no-counters` to remove).
- ☐ Connect `aclk` and `aresetn` from the system.
- ☐ Verify downstream slaves can accept backpressure.
- ☐ Run `verilator --lint-only` on the generated RTL.

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 19 Parameter Configuration

## 19.1 Generated Module Parameters

*Generated Module Parameters*

| Parameter | Default | Description |
|---|---|---|
| DATA_WIDTH | CLI --data-width | Width of TDATA in bits. |
| DEST_WIDTH | clog2(NUM_SLAVES) | Width of TDEST. |
| ID_WIDTH | clog2(NUM_MASTERS) | Width of TID. |
| USER_WIDTH | CLI --user-width | Width of TUSER. |
| NUM_MASTERS | CLI --masters | Number of master interfaces. |
| NUM_SLAVES | CLI --slaves | Number of slave interfaces. |
| STRB_WIDTH | CLI --strb-width | Width of TSTRB (only when --strb-width is used). |
| TDATACHK_W | DATA_WIDTH / 8 | Width of TDATACHK (only with --amba5). |
| TSTRBCHK_W | (STRB_WIDTH + 7) / 8 | Width of TSTRBCHK (only with --amba5 and --strb-width). |
| TIDCHK_W | (ID_WIDTH + 7) / 8 | Width of TIDCHK (only with --amba5). |
| TDESTCHK_W | (DEST_WIDTH + 7) / 8 | Width of TDESTCHK (only with --amba5). |
| TUSERCHK_W | (USER_WIDTH + 7) / 8 | Width of TUSERCHK (only with --amba5). |

## 19.2 Generator CLI Options

*Generator Command-Line Options*

| Option | Values | Default | Description |
|---|---|---|---|
| `--topology` | `flat` only | `flat` | Crossbar topology (tree/both retired 2026-10-12). |
| `--masters` | 1-32 | required | Number of master interfaces. |
| `--slaves` | 1-256 | required | Number of slave interfaces. |
| `--data-width` | 8-1024 | 64 | `TDATA` width in bits. |
| `--user-width` | 1-32 | 1 | `TUSER` width in bits. |
| `--strb-width` | ≥ 0 | 0 | `TSTRB` width; 0 omits `TSTRB`. |
| `--amba5` | flag | off | Emit AMBA 5 `TWAKEUP`, the IHI 0051B ch. 5 check signals and the per-port `*_parity_error` flags. Requires `--data-width` to be a multiple of 8. |
| `--no-counters` | flag | off | Disable packet counters. |
| `--output-dir` | path | `../rtl` | Output directory for generated SystemVerilog. |

## 19.3 Example Configurations

### 19.3.1 Base flavor 4×16 @ 64-bit

```
python3 bin/delta_generator.py \
    --topology flat --masters 4 --slaves 16 --data-width 64 \
    --output-dir rtl/
```

### 19.3.2 Rich flavor 2×4 @ 32-bit with TSTRB and AMBA5

```
python3 bin/delta_generator.py \
    --topology flat --masters 2 --slaves 4 --data-width 32 \
    --strb-width 4 --amba5 --output-dir /tmp/delta_amba5/
cp /tmp/delta_amba5/delta_axis_flat_2x4.sv
rtl_test/delta_axis_flat_2x4_strb_amba5.sv
```

The generator names its output by shape. Writing straight into `rtl_test/` would overwrite the plain `delta_axis_flat_2x4.sv`. The compile closure is `rtl_test/filelists/delta_axis_flat_2x4_strb_amba5.f`, which `-f` includes `rtl/common/filelists/dataint_parity.f`.

### 19.3.3 Base flavor without counters

```
python3 bin/delta_generator.py \
    --topology flat --masters 4 --slaves 16 --data-width 64 \
    --no-counters --output-dir rtl/
```

# 20 Verification Strategy

## 20.1 Current Status

As of 2026-10-12, Delta has one cocotb suite in `dv/tests/`. It covers the `--amba5` flavor through the checked-in example `rtl_test/delta_axis_flat_2x4_strb_amba5.sv` (2x4, 32-bit, TSTRB), using a thin flattening harness (`dv/tb/delta_axis_flat_2x4_strb_amba5_th.sv`) and RDS-DV AXIS5 BFMs on every port:

- **routing:** every master sends to every slave at once. Each slave must receive exactly its beats, every beat must pass the slave BFM's odd-parity check, the crossbar-driven TVALIDCHK/TWAKEUPCHK/TREADYCHK must equal the inversion of their signals on every cycle, and no error flag may set. At func and full the BFMs add random VALID gaps and READY stalls.
- **wakeup:** TWAKEUP with no TVALID reaches only the TDEST slave, with TWAKEUPCHK inverted.
- **chk_errors:** a bad TDATACHK, a bad TVALIDCHK and a bad TREADYCHK each set the right sticky flag and no other. Reset clears it.

Run it with `make clean-all && make run-all-func-parallel` in `dv/tests/`. The base flavor (`rtl/delta_axis_flat_4x16.sv` and the plain `rtl_test/` shapes) still has no simulation suite. It is verified by lint and review only.

## 20.2 Generator Verification

- Run `bin/delta_generator.py` for several configurations.
- Compare output against checked-in reference RTL (`rtl/delta_axis_flat_4x16.sv`, `rtl_test/delta_axis_flat_*.sv`).
- Verify that default output is byte-identical to pre-2026-10-12 generations.

## 20.3 Lint Verification

```
verilator --lint-only rtl/delta_axis_flat_4x16.sv
```

Or use the provided `rtl/Makefile`:

```
cd rtl && make verilator
```

## 20.4 Planned RTL Verification

Still to add:

- All master-to-slave routing combinations.
- Concurrent traffic to the same slave (round-robin fairness).
- Concurrent traffic to different slaves (no crosstalk).
- Backpressure stress tests.
- Packet atomicity verification (no interleaving before `TLAST`).
- Base-flavor equivalents of the `--amba5` suite (the AMBA 5 check signals are covered today).
- Reset behavior.

## 20.5 Formal Properties

When `FORMAL` is defined, the generated RTL includes:

- One-hot grant assertion per slave.
- TDEST range assertion per master.

These can be exercised with a SystemVerilog formal tool.

RTL Design Sherpa · Learning Hardware Design Through Practice  Documentation Index · MIT License

---

# 21 Synthesis and Implementation

## 21.1 Synthesis Flow

The generated module is a flat SystemVerilog file intended for standard synthesis:

```
# Example Vivado flow
read_verilog rtl/delta_axis_flat_4x16.sv
synth_design -top delta_axis_flat_4x16 -part xczu7ev-ffvc1156-2-e
```

## 21.2 Timing Considerations

- The critical path is typically from `s_axis_tvalid/tdest` through the decode and mux to `m_axis_tdata/tvalid`.
- Because the grant is registered, the mux select is stable for a full cycle once a grant is made.
- The `TREADY` path is combinational from `m_axis_tready` to `s_axis_tready`.

## 21.3 Floorplanning Suggestions

- For large crossbars, consider registering `s_axis_tdata` at the master boundary if the combinational mux path is too long.
- Keep related master/slave groups physically close to reduce routing congestion.
- If Fmax is insufficient, the generator supports `--no-counters` and reduced `DATA_WIDTH` to ease timing.

## 21.4 Implementation Checklist

- ☐ Synthesize the generated module in the target tool.
- ☐ Check setup/hold timing on `aclk`.
- ☐ Verify `aresetn` deassertion is synchronous to `aclk`.
- ☐ Run a post-implementation functional simulation if the design is modified after generation.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

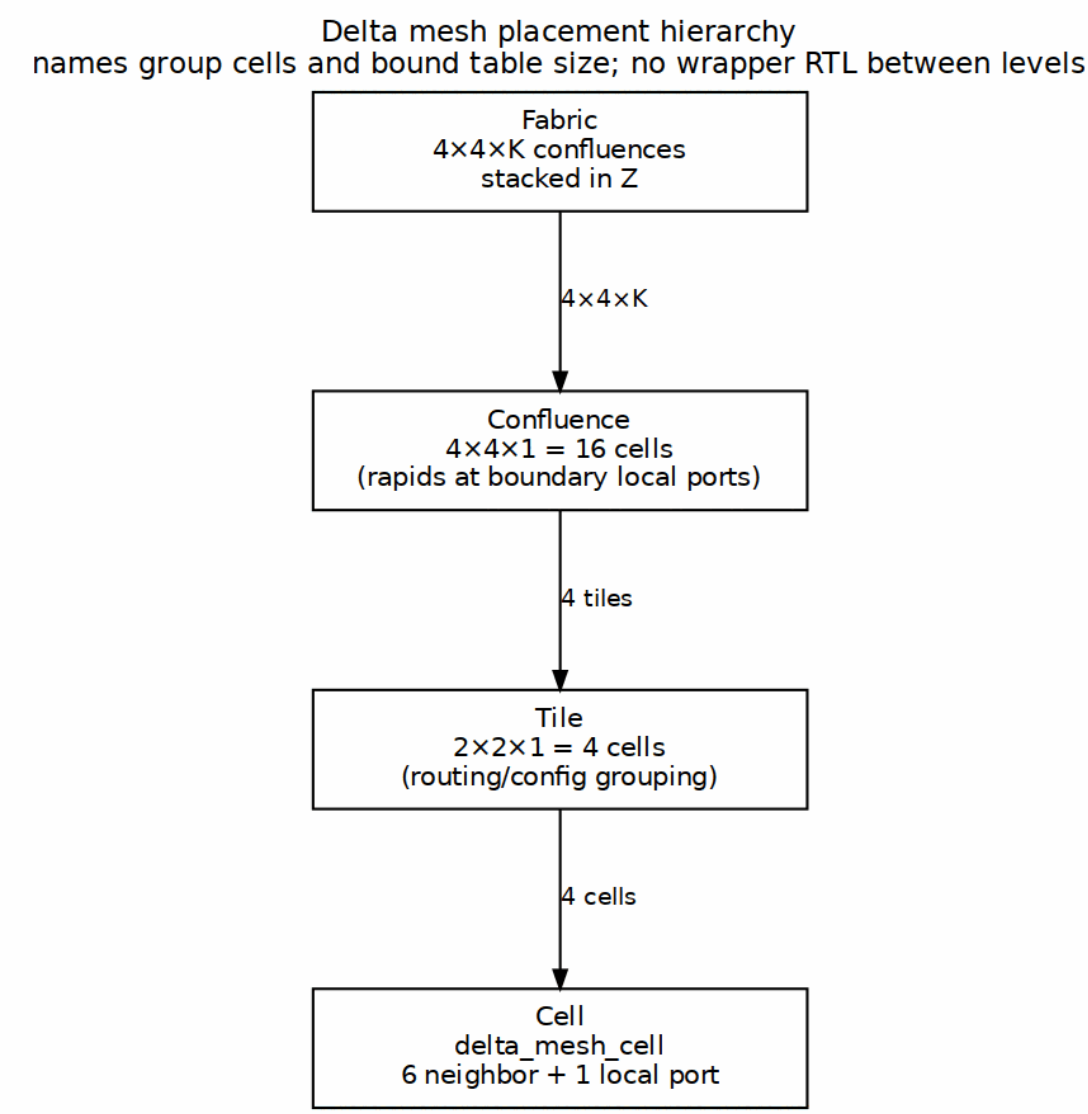
# 22 Mesh Hierarchy

## 22.1 Placement Levels

> **PLANNED** — Mesh mode is a planned generator option. The only RTL delta is expected to emit is `delta_mesh_cell`; the hierarchy below is placement and naming, not wrapper RTL. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Delta's mesh groups identical cells into four named levels so that routing tables, config images, and rapids attachment points can be named without forcing a separate module boundary at every level.

### 22.1.1 Figure 7.1: Delta Mesh Placement Hierarchy




*Delta Mesh Hierarchy*

## 22.2 Cell

A cell is `delta_mesh_cell`, the one module the generator emits for mesh mode. It has six neighbor ports (±X, ±Y, ±Z) and one local port. A cell forwards flits by looking up a destination coordinate in a RAM routing table and applying credit-based flow control per VC.

## 22.3 Tile

A tile is 2×2×1 cells — four cells arranged in X and Y, one deep in Z. Tile is a routing and config grouping: cells inside one tile are close enough that their tables can share a config burst and that the generator can number their coordinates contiguously. There is no tile wrapper; an intra-tile hop is an ordinary ±X/±Y hop.

## 22.4 Confluence

A confluence is 4×4×1 cells — sixteen cells, or four tiles. Rapids DMA attaches at the local ports of boundary cells, so a confluence is the natural unit that exposes enough boundary local ports to feed the fabric. The name was chosen by the owner on 2026-10-13. Like the tile, a confluence has no wrapper RTL; an intra-confluence hop is an ordinary hop.

## 22.5 Fabric

A fabric is 4×4×K confluences stacked in Z. The recursion is planar: the Z dimension lives between confluence layers, not inside a confluence. The generator produces a fabric top that instantiates the `cells-x × cells-y × cells-z` grid of `delta_mesh_cell`, parameterizes coordinate widths and per-dimension wrap links, and exposes boundary local ports as rapids attachment points.

## 22.6 Why This Grouping

The grouping borrows the superswitch idea seen in ECLIPSE: clustering cells so a message can reach any cell in a local group cheaply. Delta keeps the naming and table-bounding benefit but does not promise single-cycle traversal across a tile or confluence. That trade keeps the cell simple — one module, one set of tables, one set of allocators — while still letting software reason about locality.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 23 Cell and Flit

## 23.1 Cell Ports

> **PLANNED** — The mesh cell RTL (`delta_mesh_cell`) is not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Each cell has seven AXI4-Stream interfaces:

- **6 neighbor ports** — +X, −X, +Y, −Y, +Z, −Z. Wrap links for any dimension are parameterizable with `--torus=x,y,z`.
- **1 local port** — host-facing. Boundary cells expose this port for rapids DMA attachment; interior cells connect to local compute (SERV, CGRA, ocelus) outside delta's scope.

All ports carry `TDATA`, `TVALID`, `TREADY`, and `TLAST`. The head beat also carries the packet header in `TUSER`. Optional AMBA 5 sidebands (`TWAKEUP`, `TDATACHK` and the chapter 5 check signals) carry over from flat mode unchanged when `--amba5` is enabled.

## 23.2 Clock and Reset

The mesh cell uses a single clock (`aclk`) and the active-low asynchronous reset (`aresetn`) common to the rest of delta. This is a deliberate simplification *against* the GALS direction common in the NoC literature. The trade: timing closure and generator predictability win over per-link clock flexibility. If a future system needs independent clock domains, that becomes a boundary problem solved outside the cell.

## 23.3 Flit Format

A flit is one DATA_WIDTH beat. A packet is a head flit, zero or more body flits, and a TLAST tail beat. The head flit carries the header in TUSER; body and tail flits carry only payload and TLAST.

### 23.3.1 Head-Flit Header Fields

*Head-flit header fields carried in TUSER on the head beat.*

| Field | Width | Purpose |
|---|---|---|
| dest coord | 3×COORD_W | X, Y, Z destination. COORD_W is sized by the fabric dimensions. |
| src coord | 3×COORD_W | X, Y, Z source, used for responses and counters. |
| QoS class | 2 | 0 = config, 1 = latency, 2 = bulk. |
| VC | 1 | Assigned at injection per §6. |
| multicast | 1 | Set for multicast packets; group/mask reference is mode-specific. |
| misroute budget | 3 | Decremented on each non-minimal hop; packet dropped and counted at 0. |
| source-route ptr | 4 | Indexes an inline port list when source-route mode is active. |
| packet ID | 4 | Tolerates reordering at endpoints under multi-VC or adaptive routing. |

Payload streams pass transparently through the cell once the head beat has supplied the route. This is the same pass-through principle used by flat mode's TDEST handling, only the routing information now lives in TUSER rather than TDEST.

# 24 Routing and Configuration

## 24.1 Routing Tables

> **PLANNED** — The mesh cell RTL and its routing tables are not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Each `delta_mesh_cell` holds a RAM routing table. The lookup key is `(destination coordinate, QoS class)` and the result is `(output port mask, VC)`. Because QoS classes are isolated, each class can have an independent route to the same destination, giving path diversity without changing the cell datapath.

The table is a standard node-table implementation: one lookup per hop. That lookup penalty is why source-route mode (§7.3) exists for latency-critical systolic schedules.

## 24.2 Reset Default

At reset every cell loads a hardwired dimension-ordered route toward the rapids boundary. The route is connected and acyclic by construction, so it forms a safe bootstrap path even before software loads custom tables. In-band config packets ride this default route.

## 24.3 Table Constraints

The generator and the table-load checker enforce three rules that keep the mesh deadlock-free:

- VC0 entries must form a connected, acyclic subrelation in the extended channel-dependency graph. This is checked at table-load time, not just at reset.
- VC1 entries must be minimal, or must carry a misroute budget and create no indirect dependence on VC0.
- VC1→VC0 transitions are permitted at any hop; VC0→VC1 transitions are forbidden.

## 24.4 In-Band Configuration

Config packets target a reserved destination and are intercepted by the cell's ejection block instead of being delivered to the local port. The config format is `{op (read/write), entry index, entry data}`. Reads return response packets to the source so software can load-verify.

Per-cell unicast loading is the primary way to install custom routes: N cells × M entries are written at boot, and individual entries can be rewritten at runtime. Config traffic uses QoS class 0 and the reset-default route; a boundary cell or rapids injects the config stream.

## 24.5 Broadcast Mode-Switch

A broadcast mode-switch message switches the entire fabric to one of the standard modes. Every cell stores generator-emitted images for the standard modes in ROM or preloaded RAM. The new mode applies only after a fabric-wide ack barrier: each cell returns a config read-response ack, and the injector treats the mode as active only when all acks are in. No packet straddles two modes.

Mode 0 is custom — the per-cell unicast tables. Modes 1-8 are the standard fabric modes. Switching among standard modes does not touch the custom tables.

# 25 Flow Control and Deadlock

## 25.1 Credit-Based Flow Control

> **PLANNED** — The mesh cell RTL is not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Delta's mesh uses credit-based flow control per VC. Each output port tracks the number of free buffer slots in the downstream neighbor. A flit leaves only when a credit is available, and a credit returns only after the downstream cell has consumed the flit. Credit-counter depth is sized to cover round-trip latency; undersizing drops the channel duty factor below one.

## 25.2 Two Virtual Channels

The mesh has 2 VCs:

- **VC0** — the escape channel. It carries only reset-default and turn-model-conforming routes that form a connected, acyclic subrelation.
- **VC1** — the adaptive channel. It may carry minimal routes or routes with a bounded misroute budget.

This is Duato's Protocol: one adaptive channel set plus one deterministic escape set. The escape set breaks any cycle that could form in the adaptive set.

## 25.3 VC Transition Rules

A packet may move from VC1 to VC0 at any hop. VC0 to VC1 is forbidden. Because every VC1 packet can always drain into VC0, cycles in the adaptive channel cannot close into a deadlock.

## 25.4 Torus Wrap and the Dateline Rule

Per-dimension torus wrap (`--torus=x,y,z`) restores edge symmetry and halves diameter, but a wrap channel creates a cycle that plain dimension-ordered routing cannot break. For each wrapped dimension, one designated wrap channel participates in VC0 only through the escape subrelation. Other wrap traffic uses VC1 with the misroute budget. This dateline rule keeps the 2-VC budget sufficient even with wrap links.

## 25.5 Livelock

Non-minimal routing is the only source of livelock. The misroute budget in the head flit bounds it: every non-minimal hop decrements the budget, and when it reaches zero the packet is dropped and counted. Minimal routing is the normal case; the budget exists to bound VC1 alternates.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 26 Quality of Service

## 26.1 Three Traffic Classes

> **PLANNED** — The mesh cell RTL is not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Delta's mesh defines 3 QoS classes:

- **Class 0 — config/control** — highest priority; must never be blocked by data traffic.
- **Class 1 — latency** — interactive or latency-sensitive traffic.
- **Class 2 — bulk** — throughput traffic that can tolerate delay.

This split mirrors the control/data-plane priority split common in NPU practice.

## 26.2 Strict Priority With Aging

Every output port uses strict priority among the classes. To prevent starvation of lower-priority traffic, the arbiter includes programmable aging: a lower-priority request that has waited too long is temporarily promoted above a newer higher-priority request.

Aging alone is not enough for class 2 under heavy class-0/1 load. A bulk minimum reservation guarantees that some fraction of each cycle is spent serving bulk traffic.

## 26.3 Non-Interference

Each class has separate VCs and separate injection buffers. Shared buffers let tree-saturation traffic in one class block unrelated traffic even with priority bits, so delta keeps the resources separate. Config class traffic gets its own VC0 slice so control can never be bulk-blocked.

## 26.4 Per-Class Routes

The routing table is keyed by `(destination, QoS class)`, so each class can follow an independent route to the same destination. Latency class might take a low-contention path while bulk class takes the shortest physical path.

## 26.5 Within-Class Fairness

Per-flow weighted round-robin or deficit round-robin within a class is a known enhancement. It is deferred until a user needs it; the current design provides class isolation and class-level fairness only.

# 27 Multicast

## 27.1 VC0 Only

> **PLANNED** — The mesh cell RTL is not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Multicast packets use VC0 only. Restricting multicast to the escape channel keeps the multicast paths inside the same connected, acyclic subrelation used by unicast traffic, so multicast adds no new channel dependencies beyond the unicast deadlock proof.

## 27.2 BRCP-Conformed Paths

Multicast routes are table entries that produce an output port mask instead of a single output. The set of paths a multicast can take is constrained to conform to the base unicast routing (BRCP — base routing conformed path-based multicast). A cell replicates a packet only onto ports that the unicast tables would also use.

## 27.3 Source-Side Replication

When a packet must fan out, the branching cell enqueues one copy per destination port. This source-side replication discipline avoids broadcasting every packet everywhere and filtering later. Incremental allocation serves whichever masked outputs are free each cycle and releases the input only after all destinations have been served.

## 27.4 Multiple Delivery VCs

Multicast reception can deadlock even when the forward paths are correct, because multiple deliveries can compete for the same buffer. Each cell therefore provides multiple delivery VCs for multicast reception.

## 27.5 No Responses

Multicast connections do not carry responses. Responses create merge volume and ordering problems that are better handled by unicast many-to-one traffic. If software needs an acknowledgment, it sends a separate unicast gather after the multicast completes.

# 28 Systolic Support and Fabric Modes

## 28.1 Torus Wrap Links

> **PLANNED** — The mesh cell RTL is not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Each dimension can be wrapped independently with `--torus=x,y,z`. Periodic boundaries restore edge symmetry and halve diameter. The wrap links are folded so wire length doubles rather than growing with the dimension size. Deadlock freedom on wraps is handled by the dateline rule in §4.

## 28.2 Route Lookahead

The cell computes the next hop's output port one hop ahead of the flit's arrival. In steady state this collapses per-hop latency toward one cycle, which is the target for systolic repeat-cycle timing.

## 28.3 Source-Route Mode

For static systolic schedules, the injection shim can place an inline port list in the head-flit header. Intermediate cells forward without a table lookup, giving fixed-latency, table-free paths. Node-table routing remains the default for dynamic traffic.

## 28.4 Standard Fabric Modes

A broadcast mode-switch (§3) selects one of 8 standard fabric modes. Every cell stores generator-emitted images for these modes. Mode 0 is custom (per-cell unicast tables).

### 28.4.1 Standard Fabric Modes

*Standard fabric modes selected by broadcast mode-switch.*

| Mode | Name | Pattern |
|---|---|---|
| 1 | Mesh (default) | Generic packet switching using RAM tables; reset default is dimension-ordered routing. |
| 2 | Systolic-1D | Linear nearest-neighbor chains, row or column selected, optional torus ring; fixed source-route paths. |
| 3 | Wavefront-2D | 2-D diagonal ripple with skewed E/W and N/S streams; per-diagonal static tables or source routes. |
| 4 | Stencil-3D | 3-D halo/ghost-zone exchange with periodic boundaries; nearest-neighbor tables plus torus wrap on all dims. |
| 5 | Broadcast | One-to-all; multicast on VC0 with port masks to every cell. |
| 6 | Convergecast/reduce | All-to-one reduction tree; reverse-broadcast tables plus ocelus counter reduction. |
| 7 | All-to-all/transpose | Permutations such as transpose and shuffle/butterfly stages; per-stage table images. |
| 8 | Pipeline + rapids-stream | Boustrophedon serpentine chain; boundary cells stream to and from rapids DMA. |

Mode images are data, not new hardware. Adding a mode means adding an image and a verification suite.

# 29 Observability and DFT

## 29.1 Ocelus Counter Suite

> **PLANNED** — The mesh cell RTL and ocelus integration are not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

Each cell exposes a set of per-port, per-VC counters readable through the config port. The suite is informed by NPU observability practice rather than by the sparse NoC literature:

- flit counts (sent/received)
- packet counts (TLAST)
- stall cycles per VC
- arbitration win/loss per class
- queue occupancy (current and max watermark)
- injection-to-ejection latency minimum, average, maximum, and jitter
- multicast replication count
- misroute-drop count
- credit starvation events

Optional head-flit timestamps at boundary cells give end-to-end latency measurements. Counters clear on `aresetn`.

## 29.2 DFT Test Mode

Test mode configures each cell into one of three deterministic routing patterns so the mesh wiring behaves as a concatenated bus. With three patterns, stuck-at and bridge faults are testable with approximately $\lceil \log_2(n) \rceil$ patterns. The mode is controlled through the config port, and each cell drives a `test_active` output.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 30 Performance and Verification

## 30.1 Performance Model

> **PLANNED** — The mesh cell RTL is not yet implemented. The design is per the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`).

The `bin/delta_mesh_perf.py` model uses the standard interconnection-network cost model:

- **Zero-load latency** `T0 = H·tr + D/v + L/b`, where H is the hop count, tr is the per-hop router delay, D is the distance, v is the propagation speed, L is the packet length, and b is the channel bandwidth.
- **Queuing** is modeled as M/D/1 under deterministic flit service.
- **Saturation throughput** is derived from the maximum channel load γ_max, using bisection-bound and multicommodity-flow analysis. The model reports minimum delivered flow, not average.
- **Traffic patterns** include uniform random, nearest-neighbor/stencil, tornado, bit-complement, and transpose.
- **Methodology** uses infinite source queues, warm-up/measurement/drain phases, and latency measured from generation to departure.

Loaded-latency microbenchmarks follow NPU benchmarking practice. Network-calculus delay and backlog bounds for the fixed-priority classes are a phase-2 addition.

## 30.2 Verification Plan

The mesh verification plan is an 8-suite cocotb program. Each suite is a generator configuration plus a dedicated test file:

1. **Routing** — reset default DOR, programmed-table routing, table load via config port with readback verify.
2. **QoS** — strict priority order, aging inversion under saturation, bulk minimum reservation, class isolation under tree-saturation attack.
3. **Multicast** — fanout correctness, no-response rule, delivery-VC deadlock soak.
4. **Deadlock soak** — random traffic under adversarial-but-valid random tables, plus VC1 stress with VC0 escape, soaking to $N \times 10^6$ flits.
5. **Systolic** — torus nearest-neighbor ring/wavefront with measured per-hop latency, source-route fixed-latency path check, and 3-D stencil halo exchange.
6. **Counters** — every ocelus counter against a golden model.
7. **DFT mode** — three patterns route correctly; pattern count check.
8. **Perf model calibration** — measured versus modeled latency and throughput on the design-spec §10 patterns; report deltas.

Formal assists include an offline VC0-table acyclicity checker, one-hot grant assertions per output, and a TLAST-before-grant-release assertion that preserves packet atomicity. Detailed microarchitecture verification lives in the Delta MAS; the design spec dated 2026-10-13 is the source of truth for the architecture described here.

# 31 Deferred Items

**PLANNED** — The mesh cell RTL is not yet implemented. The items below are recorded from the delta mesh design spec dated 2026-10-13 (`docs/superpowers/specs/2026-10-13-delta-mesh-design.md`) and are explicitly not part of the current design.

The following features were considered and deferred:

- **GT/TDM slot-table circuits** — Ethereal/NoC slot tables, headerless GT flits, end-to-end credit flow control, and optional VOQ. Phase-2 candidate.
- **Fault-tolerant routing** — block fault regions, software absorb/reinject, and fault-status registers.
- **Barrier/superstep sync wave** — ECLIPSE-style synchronization wave; lands with the CGRA work, not delta.
- **Express channels/cubes** — little use at the target radix; skipped.
- **Logical-to-physical address mapping** — revisit if remapping or fault-recovery demands it.
- **GALS/async links** — single clock is a deliberate simplification contrary to this literature direction.
- **VOQ option** — area/throughput trade; add when a user needs it.
- **Within-class per-flow fairness (WRR/DRR)** — deferred until a user asks.
- **Flit-reservation flow control** — revisit if systolic latency targets are not met by route lookahead.
- **Reliable-mode resend/NACK** — only if a control-plane user needs lossless delivery beyond class-0 VC isolation.